

SATHYABAMA INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University)

School of Computing — Department of Computer Science and Engineering

ASSIGNMENT — SET 2

Course: Malware Analysis & Reverse Engineering / Cyber Security Elective

Programme: B.E. Computer Science and Engineering	Year / Semester: IV Year
Student Name: _____	Register No.: _____
Course Outcomes Covered: CO1 – CO6	Total Marks: 20 (2+2+2+2+6+6)
SDGs Addressed: SDG 3, SDG 4, SDG 9, SDG 16	Bloom's Levels: BTL1, BTL5, BTL6

QUESTION 1 | CO1 | BTL1 | SDG 9 | [2 Marks]

An organization delays installing operating system security updates. What security issue can this create?

Answer:

Delaying or postponing operating system security updates creates a serious vulnerability exposure window. Security patches are released by vendors (Microsoft, Apple, Linux distributions, etc.) specifically to fix known flaws that attackers already understand. When an organization does not install these updates promptly, systems remain open to exploitation of publicly documented weaknesses.

Primary security issue: Unpatched / known vulnerabilities

The core problem is that unpatched systems become easy targets for malware, ransomware, remote code execution exploits, privilege escalation, and wormable attacks. Attackers regularly scan the internet for machines missing critical patches. Famous historical examples include WannaCry and NotPetya, which spread widely by exploiting unpatched SMBv1 vulnerabilities (EternalBlue) long after fixes were available.

Related risks that follow from delayed patching:

- **Increased attack surface:** Every delayed patch leaves a known door open for intrusion, phishing payloads, and drive-by downloads.
- **Compliance and regulatory failure:** Standards such as ISO 27001, PCI-DSS, and sector guidelines expect timely patch management; delays can lead to audit failures.
- **Lateral movement inside the network:** Once one unpatched host is compromised, malware can pivot to other machines that share the same missing updates.
- **Data breach and service disruption:** Successful exploitation may result in theft of confidential data, encryption of files (ransomware), or downtime of critical services.

In summary, delaying OS security updates creates a preventable security gap: systems remain vulnerable to known exploits, raising the probability of malware infection, data loss, and large-scale compromise. Regular, tested patch management aligned with SDG 9 (resilient infrastructure) is essential for organizational cyber resilience.

Recommended organisational practice: maintain an inventory of assets, classify patches by severity (critical/important), test updates in a staging environment, then deploy within a defined SLA (for example, 48–72 hours for critical internet-facing systems). Where immediate reboot is difficult, use compensating

controls such as IDS/IPS signatures, virtual patching on WAFs, and network segmentation until the OS update is applied.

QUESTION 2 | CO2 | BTL6 | SDG 16 | [2 Marks]

A malware analyst needs to inspect file hashes, running processes, registry changes, and network traffic. Discuss why a malware analysis toolkit is important.

Answer:

A malware analysis toolkit is a curated collection of static and dynamic analysis utilities that helps an analyst systematically examine suspicious software. When the task includes inspecting file hashes, running processes, registry changes, and network traffic, a toolkit is important for the following reasons:

- **Complete visibility across artefacts:** No single tool covers every indicator. Hashing utilities (MD5/SHA-256) identify known samples; process monitors reveal injected or hidden processes; registry watchers detect persistence; packet analyzers expose C2 (command-and-control) communication. A toolkit brings these views together.
- **Speed and repeatability:** Standard tools (PEview, Detect It Easy, Process Monitor, Process Explorer, Regshot, Wireshark, FakeNet, IDA/Ghidra, x64dbg, Cuckoo/sandbox) allow consistent workflows so findings can be reproduced and documented for incident response and law-enforcement reporting (supporting SDG 16 — strong institutions).
- **Safe isolation:** Toolkit components are typically used inside VMs or sandboxes, so analysts can observe behaviour without infecting production systems.
- **Correlation of evidence:** Hash matches, process trees, registry keys, and network IOCs can be correlated to classify the malware family, map kill-chain stages, and recommend blocking rules for firewalls and EDR products.

Therefore, a malware analysis toolkit is indispensable because it enables structured, multi-layer inspection of hashes, processes, registry, and traffic—turning raw observations into actionable intelligence for detection, containment, and prosecution support.

QUESTION 3 | CO3 | BTL6 | SDG 4 | [2 Marks]

While debugging a C program, you notice that local variables are accessed using offsets from the stack pointer. Discuss why offsets are used.

Answer:

In compiled C programs, local (automatic) variables, function parameters, saved registers, and return addresses live in a per-function region of memory called a **stack frame**. The CPU maintains a stack pointer register (ESP/RSP on x86/x64, or SP on ARM) that always points to the current top of the stack. Compilers access locals using **constant offsets from this pointer** (or from a frame pointer such as EBP/RBP) for several fundamental reasons:

- **Relative addressing is efficient:** Absolute addresses of locals are not known at compile time because the stack grows and shrinks at runtime as functions call and return. Offsets from SP/BP give a fixed, cheap way to locate each variable within the current frame (e.g., `[rbp-0x8]` for an integer).
- **Frame layout is deterministic:** The compiler allocates space for each local at a known displacement when the function prologue runs (`sub rsp, N`). Using offsets keeps load/store instructions compact and predictable.
- **Supports recursion and concurrency:** Each invocation has its own stack frame. Offsets relative to that frame's pointer ensure that recursive or multi-threaded calls do not overwrite one another's locals.

- **Debugging and reverse engineering insight:** Analysts reading disassembly recognize patterns such as `mov eax, [rbp-4]` as local variable access—critical when reversing malware written in C/C++.

Hence, offsets from the stack (or frame) pointer are used because locals have no fixed absolute address; the offset model is compact, correct across nested calls, and central to understanding how high-level C maps to machine code during debugging and malware reverse engineering (aligned with SDG 4 — quality technical education).

QUESTION 4 | CO4 | BTL5 | SDG 16 | [2 Marks]

A suspicious calculator application crashes while executing malicious code. Choose which debugging mode is appropriate?

Answer:

The appropriate debugging mode in this scenario is **post-mortem debugging** (also called **crash dump / dump analysis** mode), often combined with **user-mode debugging** when the process can still be attached. Because the suspicious calculator crashes while malicious code is running, a live interactive session may be lost at the moment of failure; post-mortem analysis recovers the state at crash time.

Why post-mortem / crash-dump debugging is appropriate:

- When malware triggers an access violation, heap corruption, or intentional crash (anti-analysis), the OS can generate a minidump or full memory dump. Tools such as WinDbg, OllyDbg/x64dbg (with exception handling), or Volatility on a memory image let the analyst inspect registers, call stack, loaded modules, and the faulting instruction after the crash.
- If the sample is still running but unstable, **user-mode debugging** (attaching x64dbg/OllyDbg to the calculator process) with first-chance exception breakpoints is also suitable to catch the fault before termination.
- **Kernel-mode debugging** is generally unnecessary for a user-space calculator unless a malicious driver is involved.

Comparative note:

Live (interactive) debugging is preferred when the program stays alive; post-mortem debugging is preferred when it terminates abruptly. For a crashing malicious calculator, start with exception breakpoints in a user-mode debugger inside a VM; if the process dies instantly, collect and analyse a crash dump. This approach supports reliable evidence collection for security investigations (SDG 16).

QUESTION 5 | CO5 | BTL6 | SDG 9 | [6 Marks]

A newly discovered malware sample exhibits no suspicious behaviour during initial execution but activates only after specific user actions. Design a dynamic analysis environment using sandboxes, packet analyzers, debuggers, and monitoring tools to uncover the malware’s behaviour. Justify your design choices.

Answer:

5.1 Problem understanding

The sample is likely **user-action triggered**, **environment-aware**, or **sandbox-evasive**. It may wait for clicks, document opens, delayed timers, presence of user files, or mouse movement before decrypting its payload. A naïve “run for 30 seconds in a sandbox” will miss the behaviour. The lab must therefore combine automated dynamic analysis with interactive stimulation and deep monitoring.

5.2 High-level architecture of the analysis environment

The proposed environment is an **isolated, multi-VM dynamic analysis lab** with the following layers:

- **Host / hypervisor:** Type-2 hypervisor (VMware Workstation or VirtualBox) on a dedicated analysis machine with host-only / custom NAT networking; snapshots before every run; no shared folders to production data.
- **Guest analysis VMs:** Windows 10/11 and optionally Windows 7 x86/x64 with common user applications (Office, browsers, PDF readers), realistic user profile data, and disabled unnecessary cloud sync—so the malware “believes” it is on a real workstation.
- **Network simulation VM:** Separate Linux VM running INetSim / FakeNet-NG / Remnux services to fake DNS, HTTP(S), SMTP, and C2 endpoints without allowing real internet egress (or controlled proxy egress with full logging).
- **Optional automated sandbox:** Cuckoo Sandbox or CAPE Sandbox for baseline behaviour reports, extended with custom simulation scripts for mouse/keyboard events.

5.3 Tool stack and roles

(A) Sandboxes — initial behavioural triage

Use Cuckoo/CAPE or a commercial sandbox to obtain process trees, dropped files, mutexes, and API traces. Because the malware may be idle at first, configure **extended analysis time** (several minutes), **human interaction modules** (simulated clicks, typing, opening documents), and multiple OS profiles. Justification: sandboxes scale well for first-pass IOC extraction and family tagging.

(B) System monitoring tools — detect dormant-then-active changes

- Process Monitor (Procmon) with boot/advanced filters for process, file, registry, and network events.
- Process Explorer / Process Hacker for live process trees, handles, and DLLs.
- Regshot or Autoruns for before/after persistence comparison.
- Sysmon (with a tuned config) for durable event logs of process creation and network connects.

Justification: user-triggered malware often writes Run keys, scheduled tasks, or injects into explorer.exe only after interaction; continuous monitors catch the moment of activation.

(C) Packet analyzers — uncover delayed C2

- Wireshark or tcpdump on the virtual switch / host-only adapter.
- FakeNet-NG or INetSim to answer DNS/HTTP so the malware proceeds past network checks.
- Optional mitmproxy for TLS inspection with installed lab CA (when ethically and legally appropriate inside the isolated lab).

Justification: dormant samples may beacon only after a trigger; capturing DNS queries, HTTP POSTs, and unusual ports reveals C2 domains and exfiltration patterns.

(D) Debuggers — force and understand the trigger path

- x64dbg / OllyDbg for user-mode stepping; set breakpoints on GetAsyncKeyState, GetCursorPos, NtDelayExecution, CreateFile, window-message handlers, and anti-VM checks.
- IDA Pro / Ghidra for static mapping of trigger conditions before dynamic runs.
- API Monitor to log high-level API sequences during interaction.

Justification: when sandboxes show “nothing happened,” a debugger finds sleep loops, sandbox checks, and the exact user action that decrypts the payload.

5.4 Analysis workflow (step-by-step)

- **Step 1 — Safe intake:** Compute SHA-256 hash; submit hash to VirusTotal/MalwareBazaar for reputation (without uploading if policy forbids); store sample encrypted.
- **Step 2 — Static preview:** PE headers, imports, strings, packer detection (DIE, PEiD), and disassembly of likely trigger functions.

- **Step 3 — Baseline snapshot:** Clean VM snapshot; start Procmon, Wireshark, Sysmon, and FakeNet before execution.
- **Step 4 — Passive run:** Execute sample with no interaction for a short window to confirm dormancy and record any anti-analysis checks.
- **Step 5 — Stimulated run:** Perform realistic user actions—open calculator UI buttons, type text, open Office/PDF files, browse dummy intranet pages, idle with mouse movement, wait beyond common sleep thresholds (e.g., >5–10 minutes).
- **Step 6 — Debugger-assisted trigger:** If still quiet, attach debugger, patch or skip anti-VM/sleep, and force execution of suspicious branches; note decrypted strings and injected code.
- **Step 7 — Correlate & report:** Merge process, registry, file, and network timelines; extract IOCs (hashes, domains, mutexes, persistence keys); restore VM snapshot.

5.5 Expected behavioural indicators after trigger

Once the malware activates due to a specific user action, the monitoring stack should be ready to capture typical post-trigger artefacts. These indicators help confirm that the dormant phase has ended and that the payload is executing:

- **Process artefacts:** child processes spawned from the calculator (cmd.exe, powershell.exe, rundll32.exe, mshta.exe), process hollowing, or injection into trusted processes such as explorer.exe or svchost.exe.
- **File system artefacts:** dropped DLLs/EXEs in %AppData%, %Temp%, or ProgramData; modification of existing binaries; creation of batch/VBS scripts.
- **Registry / persistence:** Run / RunOnce keys, Winlogon Shell/Userinit changes, scheduled tasks, services, or COM hijacking entries appearing only after interaction.
- **Network artefacts:** sudden DNS lookups for newly registered domains, HTTPS beacons with unusual JA3 fingerprints, raw TCP to high ports, or ICMP tunnelling.
- **Anti-analysis remnants:** checks for VMware/VirtualBox MAC prefixes, CPUID hypervisor bits, short uptime, or missing recent documents—explaining why the first idle run looked benign.

5.6 Safety, ethics, and lab hygiene

Dynamic analysis of live malware must never be performed on production networks. Snapshots must be restored after every experiment; samples must be password-zipped during storage and transfer; and outbound internet should be blocked or tightly proxied. Screenshots, Procmon PML files, PCAP captures, and debugger notes form the evidence pack for the assignment report and for any institutional SOC hand-off.

5.7 Justification summary

Sandboxes alone fail against interaction-aware malware; monitors alone lack deep code insight; packet captures alone miss host-only changes; debuggers alone do not scale. Combining all four creates a resilient dynamic analysis environment that (1) isolates risk, (2) simulates a believable user workstation, (3) records host and network side effects around the trigger moment, and (4) allows forced exploration of hidden paths. This design strengthens digital infrastructure defence capability (SDG 9) and yields reproducible engineering evidence for complex malware cases (HOTS / complex problem solving).

QUESTION 6 | CO6 | BTL6 | SDG 3 & SDG 9 | [6 Marks]

A smart healthcare system stores patient records across multiple cloud platforms. Design an intelligent malware detection framework combining machine learning, signature-based detection, and

behaviour-based analysis. Evaluate its effectiveness against polymorphic and metamorphic malware.

Answer:

6.1 Context and security requirements

A smart healthcare system handling Electronic Health Records (EHRs) across multi-cloud platforms (e.g., hospital private cloud + AWS/Azure + telemedicine SaaS) must protect confidentiality, integrity, and availability of highly sensitive patient data. Malware risks include ransomware encrypting clinical databases, info-stealers exfiltrating PHI (Protected Health Information), trojans on medical IoT gateways, and supply-chain compromises of cloud-connected apps. Detection must work across endpoints (clinician workstations), servers, containers, and cloud workloads while respecting low false positives—because blocking a legitimate clinical application can endanger care (SDG 3).

6.2 Framework architecture (hybrid intelligent detection)

The proposed framework is a **layered, defence-in-depth pipeline** that fuses signature, behaviour, and machine-learning detectors behind a central correlation and response engine.

Layer 1 — Data collection agents

Lightweight sensors on endpoints, VMs, and cloud workloads collect: file hashes and PE metadata; process creation trees; registry/persistence events; DNS/HTTP telemetry; API call sequences; container syscalls (e.g., via Falco); cloud audit logs (CloudTrail / Azure Activity Log). Telemetry is shipped to a secure SIEM / data lake with encryption in transit and at rest.

Layer 2 — Signature-based detection (fast known-threat filter)

Traditional AV / YARA / ClamAV / cloud malware feeds match known hashes, byte patterns, and packer stubs. EDR blocklists and threat-intelligence IOCs (domains, IPs, certificates) are applied at email gateways, web proxies, and cloud CASB controls. *Role:* near-zero-latency blocking of commodity malware and known ransomware families before they touch EHR stores.

Layer 3 — Behaviour-based analysis (dynamic & heuristic)

Sandbox detonation of unknown attachments/binaries; behavioural rules for ransomware-like mass file rename/encryption, credential dumping (LSASS access), unusual PowerShell / WMI, lateral movement via SMB/RDP, and abnormal database bulk exports. Cloud behaviour rules flag sudden mass object downloads from FHIR/EHR buckets or privilege escalation in IAM roles. *Role:* catch novel samples whose bytes are unknown but whose actions are malicious.

Layer 4 — Machine learning detection (adaptive intelligence)

- **Static ML:** gradient boosting / neural models on PE features (entropy, section sizes, import hashes, opcode n-grams) to score unknown binaries.
 - **Dynamic ML:** sequence models (LSTM/Transformer) or graph ML on process–file–network event graphs.
 - **Anomaly detection:** unsupervised models (autoencoders, Isolation Forest) on user and service baselines—e.g., a radiology workstation suddenly beaconing to a rare domain at 3 a.m.
 - **NLP on scripts:** classify obfuscated PowerShell/JS often used in healthcare phishing.
- Models are trained on labelled malware corpora plus healthcare-specific benign software (PACS viewers, LIS clients) to reduce false positives. Continuous retraining with analyst feedback (human-in-the-loop) is mandatory.

Layer 5 — Correlation, scoring, and automated response

A SOAR/XDR brain aggregates alerts with weighted risk scores. High-confidence detections auto-isolate

the host, revoke cloud tokens, snapshot affected volumes, and open tickets for the SOC. Medium-confidence items queue for analyst review. Immutable backups and network micro-segmentation limit blast radius across clouds.

6.3 Multi-cloud healthcare deployment considerations

- Unified agent policy across on-prem and cloud VMs; container image scanning in CI/CD before deployment to Kubernetes clusters hosting FHIR APIs.
- Privacy-preserving telemetry: minimise PHI in logs; tokenize identifiers; comply with HIPAA/DISHA-equivalent local regulations and hospital ethics boards.
- High availability: detection control plane must not become a single point of failure for clinical operations; fail-open vs fail-closed decisions are risk-assessed per tier.
- Zero-trust access: MFA, least privilege, and continuous device posture checks for clinicians accessing multi-cloud records.

6.4 Evaluation against polymorphic and metamorphic malware

Polymorphic malware mutates its decryptor / appearance on each generation while keeping a similar encrypted payload body. Pure signature matching often fails because hash and byte signatures change. In the proposed framework:

- Signature layer remains useful for known decryptor stubs and shared C2 IOCs but is **insufficient alone**.
- Behaviour layer remains strong: encryption behaviour, process injection, and C2 patterns are largely stable across polymorphic variants.
- ML static features (opcode histograms, import graphs) and dynamic sequence models generalise across cosmetic mutations—**effectiveness: high** when models are retrained on fresh variants.

Metamorphic malware rewrites its own code (permutation, junk insertion, register renaming, control-flow flattening) so that successive copies are functionally similar but structurally dissimilar. This is harder:

- Classic signatures and simple static ML features degrade significantly.
- Behaviour-based analysis and dynamic ML on API/system-call graphs retain the best signal because semantics of malicious goals (ransom, steal, lateral move) persist.
- Graph-based ML, sandbox detonation, and memory analysis (detecting unpacked payloads in RAM) are critical; effectiveness is **moderate to high** with layered controls, but not perfect—advanced metamorphic threats may still require skilled reverse engineering.

6.5 Overall effectiveness assessment

Technique	Known malware	Polymorphic	Metamorphic
Signatures / YARA / hash	Excellent	Weak–Fair	Poor
Behaviour / sandbox / heuristics	Good	Good–Excellent	Good
Machine learning (static+dynamic)	Excellent	Good–Excellent	Fair–Good
Hybrid framework (proposed)	Excellent	Excellent	Good–Excellent*

**With continuous model updates, memory analysis, and rapid SOC response.*

For a multi-cloud smart healthcare system, a hybrid framework that combines signature speed, behavioural semantics, and adaptive ML provides the strongest practical defence against polymorphic and metamorphic

malware. Signatures stop the known; behaviour and ML stop the unknown and the shape-shifting. Coupled with zero-trust cloud controls and immutable backups, the design protects patient safety and data (SDG 3) while hardening resilient digital health infrastructure (SDG 9).

6.6 Illustrative detection scenario (ransomware on EHR node)

Consider a clinician workstation that downloads a malicious “invoice” macro document. The signature engine may miss a brand-new polymorphic dropper. Behavioural rules detect Office spawning PowerShell with encoded commands; the ML model scores the script high for obfuscation; sandbox detonation shows mass file encryption patterns. The correlation engine isolates the host, blocks the C2 domain at the cloud DNS firewall, and verifies that immutable EHR backups in a second cloud region remain untouched. This end-to-end story shows why a single detection method is inadequate in healthcare settings.

6.7 Limitations and future improvements

- **Adversarial ML:** attackers may craft samples to evade classifiers; adversarial training and ensemble models are needed.
- **Encrypted C2 / living-off-the-land:** behaviour rules must focus on intent (unusual access to medical databases) rather than only known malware binaries.
- **Alert fatigue:** tuning thresholds with clinical workflow owners reduces false positives that distract SOC analysts.
- **Future work:** federated learning across hospitals (without sharing raw PHI), memory-only detection for fileless threats, and tighter integration with medical device (IoMT) anomaly monitoring.

*Final evaluation: The hybrid intelligent framework is **highly effective** against polymorphic malware and **substantially effective** against metamorphic malware when behaviour and dynamic ML layers are prioritised. It meets the engineering need to protect multi-cloud patient records without relying on any single detection theory.*

COURSE OUTCOME & SDG MAPPING SUMMARY

The following table summarises how each answer addresses the stated course outcomes, Bloom’s taxonomy levels, and Sustainable Development Goals for this SET 2 assignment.

Q	CO	BTL	SDG	Key learning demonstrated
1	CO1	BTL1	9	Identify risk from delayed OS patching / unpatched vulnerabilities
2	CO2	BTL6	16	Justify importance of an integrated malware analysis toolkit
3	CO3	BTL6	4	Explain stack-pointer offsets for locals in C debugging
4	CO4	BTL5	16	Select post-mortem / user-mode debugging for crashing malware
5	CO5	BTL6	9	Design dynamic lab for user-action-triggered malware
6	CO6	BTL6	3, 9	Design hybrid ML+signature+behaviour detection for healthcare cloud

REFERENCES (Suggested)

1. Sikorski, M. & Honig, A. — *Practical Malware Analysis*, No Starch Press.
2. Monnappa, K. A. — *Learning Malware Analysis*, Packt Publishing.
3. Eagle, C. & Nance, K. — *The Ghidra Book*, No Starch Press.
4. Microsoft Security — Windows debugging, Sysinternals Suite, and Microsoft Defender documentation.
5. NIST SP 800-83 Rev. 1 — Guide to Malware Incident Prevention and Handling.

6. NIST SP 800-61 Rev. 2 — Computer Security Incident Handling Guide.
7. ENISA — Cybersecurity guidelines for the health sector.
8. WHO / national e-health security advisories on protecting patient data systems.
9. MITRE ATT&CK framework — adversary tactics and techniques for behaviour mapping.

KEY TAKEAWAYS FOR MALWARE ANALYSIS PRACTICE

- Keep systems patched—known vulnerabilities are the easiest entry path for malware.
- Use a multi-tool toolkit so hash, process, registry, and network views can be correlated.
- Understand stack frames and offsets—they are the language of low-level debugging and RE.
- Match the debugging mode to the symptom: crashing malware often needs post-mortem dumps.
- Dormant, user-triggered samples require stimulated dynamic analysis, not a short idle sandbox run.
- Healthcare multi-cloud defence needs hybrid detection: signatures + behaviour + ML, with patient-safety-aware response.

Declaration: This assignment is submitted as part of the B.E. CSE (IV Year) curriculum at Sathyabama Institute of Science and Technology. The answers demonstrate understanding of malware analysis concepts mapped to CO1–CO6, Bloom’s taxonomy levels indicated in SET 2, and the relevant Sustainable Development Goals (SDG 3, 4, 9, and 16).

Student Signature: _____

Date: _____

Faculty In-charge: _____

Marks Awarded: _____ / 20